

Mémo technique

Personal Finance Tracker

Comprendre et défendre le projet, du plus simple au plus avancé

DATA 333 – Data Management & Analysis — Bellevue College

Student Name : _____

October 1, 2026

Table des matières

1	Le projet en bref	3
1.1	L'objectif	3
1.2	Les fonctionnalités	3
1.3	Le parcours d'un utilisateur	3
1.4	Les trois modes et le tableau de bord	3
2	Les concepts du cours, du plus simple au plus avancé	6
2.1	Entrées et sorties	6
2.2	Les décisions (<code>if / elif / else</code>)	7
2.3	Les boucles (<code>while, for</code>)	7
2.4	Les fonctions	7
2.5	Les listes	8
2.6	Les dictionnaires	8
2.7	Les ensembles (<code>set</code>)	8
2.8	Les classes et la programmation orientée objet	8
2.9	La gestion des fichiers	9
2.10	Les exceptions	10
2.11	pandas	11
3	Les bibliothèques	11
3.1	pandas (3.0.6)	11
3.2	matplotlib (3.11.2)	12
3.3	Plotly (7.1.0) et Streamlit (1.64.0)	12
3.4	Tkinter et ttk (Tk 8.6)	13
3.5	json, csv, pathlib (bibliothèque standard)	14
3.6	dataclasses, uuid, tempfile, argparse (bibliothèque standard)	14
3.7	pytest (9.1.1), pytest-cov (7.1.0) et ruff (0.16.9)	14
3.8	Outils des livrables : rich (15.0.0), Playwright (1.63.0), python-docx, python-pptx	14
4	Le travail en détail, module par module	15
4.1	Vue d'ensemble et flux des données	15
4.2	<code>models.py</code> — les fiches de base	15
4.3	<code>exceptions.py</code> — les erreurs du projet	16
4.4	<code>storage.py</code> — lire et écrire les fichiers	16
4.5	<code>tracker.py</code> — le classeur central	17
4.6	<code>analytics.py</code> — les calculs pandas	18
4.7	<code>alerts.py</code> — budgets et jalons	18
4.8	<code>visualize.py</code> — les graphiques	19
4.9	<code>config.py</code> — les chemins	20
4.10	<code>cli.py</code> — le menu console	20

4.11	<code>gui_tkinter.py</code> — l'application de bureau	20
4.12	<code>dashboard.py</code> — le tableau de bord	21
4.13	<code>main.py</code> — le point d'entrée	21
5	Les données	21
5.1	Le format CSV	21
5.2	Le format JSON	22
5.3	Pourquoi ces choix	22
5.4	La génération des données d'exemple	22
6	Les tests	23
6.1	Ce qui est testé, et pourquoi	23
6.2	Comment les lancer	23
7	Commandes utiles	23
8	Les 15 questions probables de l'évaluateur	24
9	Glossaire	26

1 Le projet en bref

1.1 L'objectif

Le *Personal Finance Tracker* est une application Python qui sert à suivre son argent : on y enregistre ses revenus et ses dépenses, on les classe par catégorie, et l'application transforme ces données en résumés, en tendances, en alertes de budget et en suivi d'objectifs d'épargne. Elle répond au cahier des charges du cours DATA 333 (suivre revenus et dépenses, catégoriser, résumer, objectifs d'épargne, fichiers, tendances, graphiques) et implémente les quatre améliorations demandées : interface Tkinter, stockage JSON et CSV, alertes de budget et tableau de bord Streamlit.

À retenir pour l'oral

Une phrase pour commencer l'oral : « Un seul cœur testé (modèles, stockage, analyses, alertes) et trois façons de s'en servir : la console, une application de bureau et un tableau de bord web. »

1.2 Les fonctionnalités

- **Transactions** : ajouter, modifier, supprimer, rechercher par texte, filtrer (type, catégorie, période, montant). Chaque valeur est vérifiée avant d'être gardée.
- **Résumés et tendances** : revenus, dépenses, solde et taux d'épargne par mois ; part de chaque catégorie ; moyenne glissante sur 3 mois ; variation d'un mois à l'autre ; plus grosses dépenses.
- **Budgets et alertes** : une limite mensuelle par catégorie ; *WARNING* à 80 %, *EXCEEDED* à 100 %, affichée dès qu'une dépense franchit un seuil.
- **Objectifs d'épargne** : montant cible, échéance, versements, jalons à 25/50/75/100 %.
- **Fichiers** : tout l'état en JSON, les transactions en CSV (import/export), écriture atomique, récupération d'un fichier corrompu.
- **Graphiques** : camembert par catégorie, barres revenus/dépenses, courbe de tendance, progression des objectifs (PNG), plus des graphiques interactifs Plotly.

1.3 Le parcours d'un utilisateur

1. Il lance l'application (console, fenêtre ou navigateur). Au premier lancement, les données d'exemple (311 transactions fictives, de 2026-04 à 2026-09) sont chargées pour qu'il ait quelque chose à explorer.
2. Il ajoute une dépense : type, montant, date, catégorie, description. Une erreur de saisie est expliquée et redemandée.
3. Si la dépense fait dépasser 80 % ou 100 % d'un budget, une alerte apparaît tout de suite.
4. Il consulte le résumé du mois, la tendance, les graphiques, ses objectifs.
5. Il quitte : les données sont sauvegardées dans `data/my_finances.json` (fichier ignoré par git, pour ne jamais publier de données personnelles).

1.4 Les trois modes et le tableau de bord

Mode console — `python main.py cli`. Un menu numéroté. Toute la saisie passe par une seule méthode (`ConsoleApp.ask`) et des boucles redemandent la valeur tant qu'elle est invalide (figure 1).

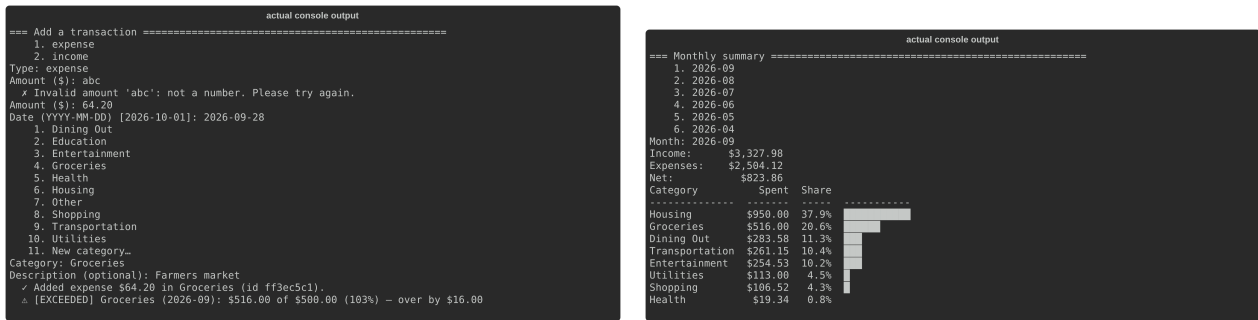


Figure 1 — Mode console — sortie réelle (*actual console output*) d'une session `python main.py cli` dont les réponses ont été tapées dans un pseudo-terminal.

Mode démonstration — `python main.py demo`. Aucune question : le programme charge les données d'exemple, affiche tous les résumés et alertes, puis écrit les 4 graphiques PNG et des exports JSON/CSV dans `output/`. C'est le mode à montrer en premier : une seule commande montre tout (figure 2).

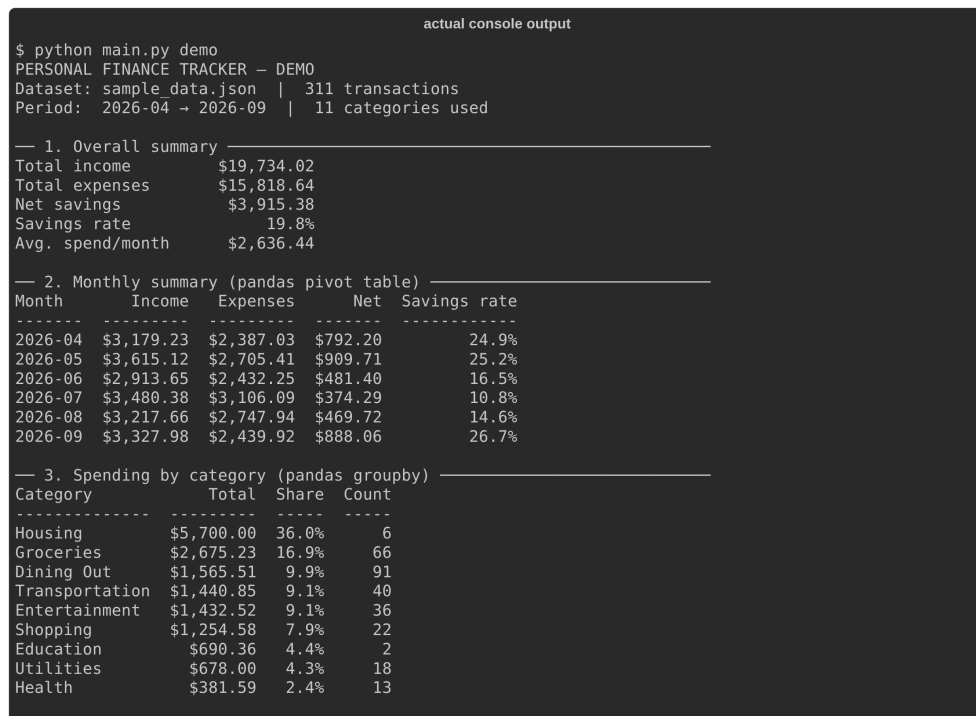


Figure 2 — Début de la sortie réelle de `python main.py demo` : résumé global, tableau mensuel (pandas `pivot_table`) et dépenses par catégorie (`groupby`).

Mode fenêtre — `python main.py gui`. Une application Tkinter à quatre onglets : Transactions, Summary, Charts, Budgets & Goals (figures 3 et 4).

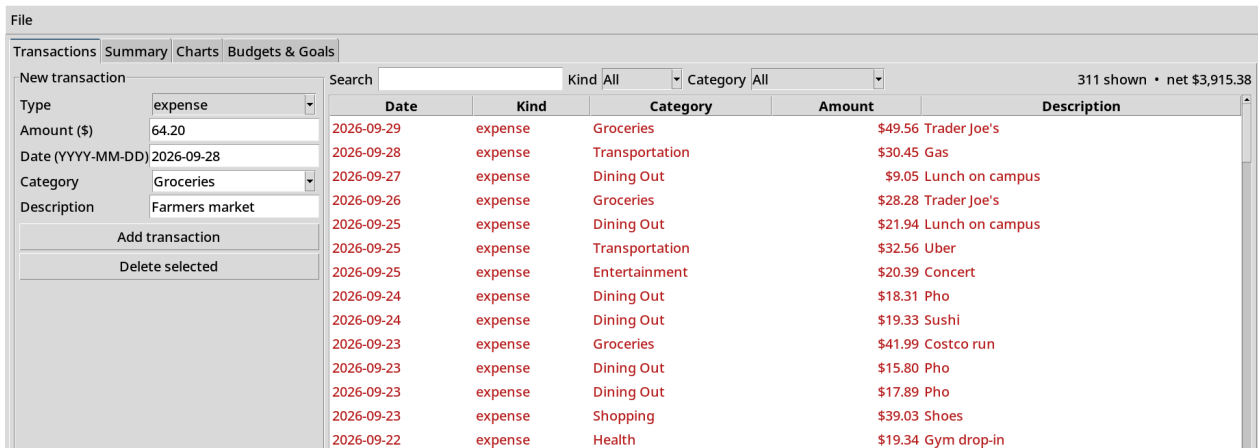
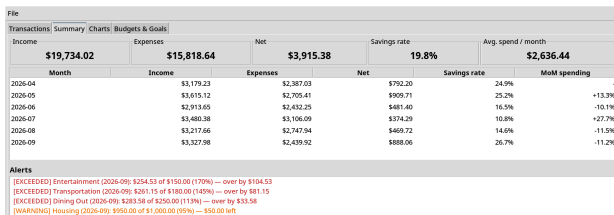
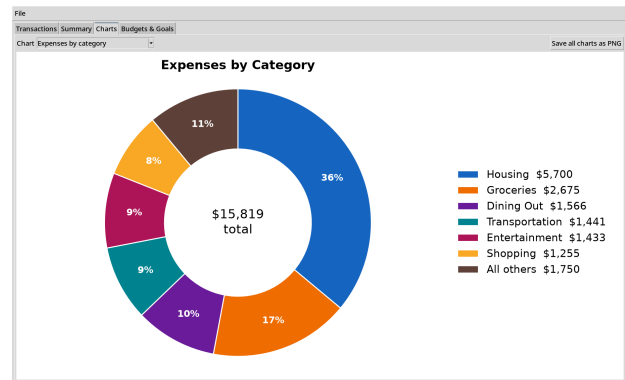


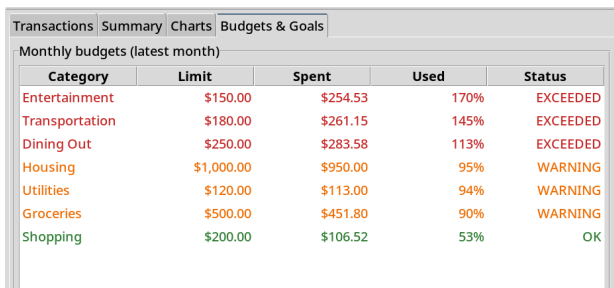
Figure 3 — Onglet Transactions : formulaire d'ajout rempli (à gauche), recherche et filtres (en haut), tableau triable par colonne. Revenus en vert, dépenses en rouge.



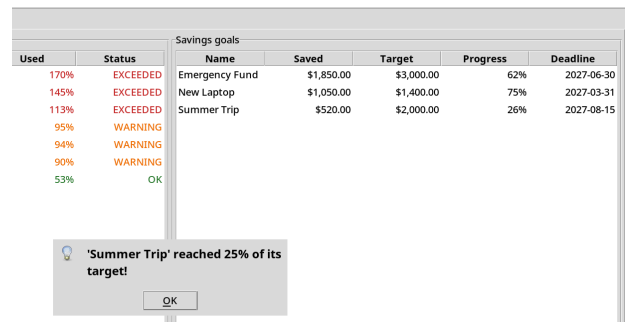
(a) Summary : indicateurs, tableau mensuel, alertes colorées.



(b) Charts : le même graphique matplotlib que le mode démo.



(c) Budgets : EXCEEDED en rouge, WARNING en orange, OK en vert.



(d) Objectifs : un versement franchit le jalon de 25 %.

Figure 4 — Les autres onglets de l'application Tkinter (captures réelles sous Xvfb).

Tableau de bord — `streamlit run finance_tracker/dashboard.py`. Une page web interactive : on choisit la source de données (exemple, données sauvegardées ou fichier importé), on filtre par période, catégories et type, et tout se recalcule (figure 5).

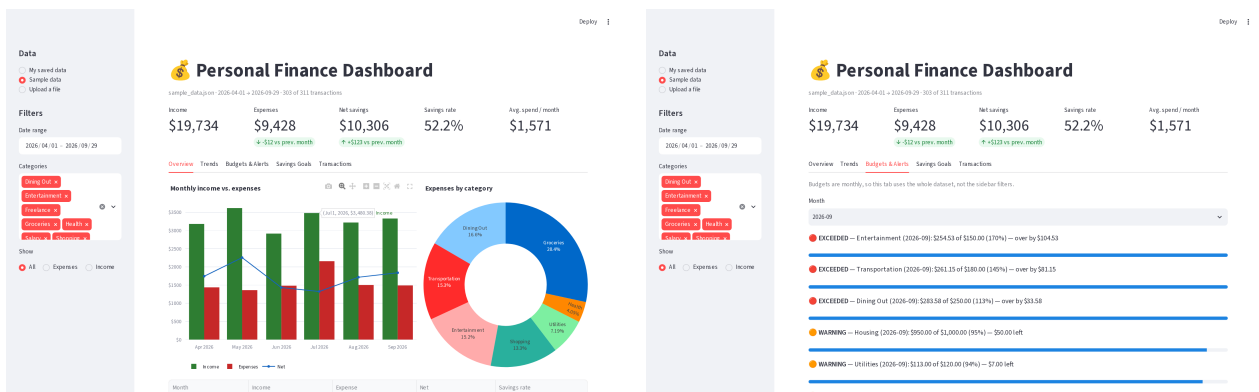
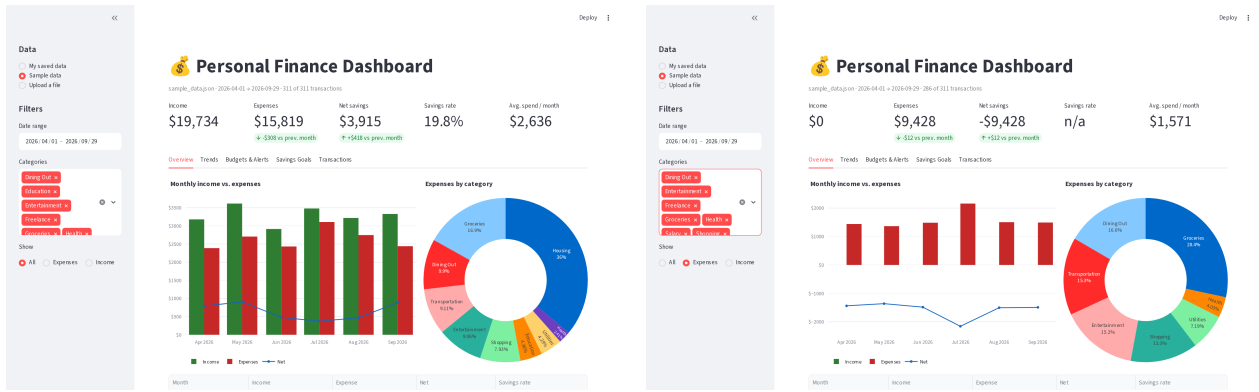


Figure 5 — Tableau de bord Streamlit, capturé avec Chromium sans affichage (Playwright).

2 Les concepts du cours, du plus simple au plus avancé

Pour chaque concept : une définition simple, une analogie, puis l'endroit exact du code où il est utilisé. Dans le code, chaque concept est marqué par un commentaire `# Concept:` (`grep -rn "# Concept:" finance_tracker` les liste tous).

2.1 Entrées et sorties

Définition simple

Une *entrée* est une information que le programme reçoit (ce que l'utilisateur tape) ; une *sortie* est ce qu'il affiche ou écrit. En Python : `input()` et `print()`.

Analogie de la vie courante

Un guichet de banque : le client donne une information, le guichetier répond avec un reçu clair et bien aligné.

Dans le projet, *toutes* les lectures clavier passent par `ConsoleApp.ask`, et les fonctions de lecture et d'écriture sont *injectées* dans la classe : les tests peuvent donc remplacer le clavier par une liste de réponses préparées. Les tableaux sont alignés avec des f-strings (`f"{montant:>12}"`).

2.2 Les décisions (if / elif / else)

Définition simple

Une structure de décision choisit quoi faire selon une condition : « si ... sinon si ... sinon ... ».

Analogie de la vie courante

Un feu tricolore : vert sous 80 % du budget, orange entre 80 et 100 %, rouge au-delà.

Extrait 1 – alerts.py — classer l'utilisation d'un budget

```
def budget_level(ratio: float) -> str:
    # Concept: decision structure -- thresholds are tested from most to least severe
    if ratio >= EXCEEDED_THRESHOLD:
        return LEVEL_EXCEEDED
    elif ratio >= WARNING_THRESHOLD:
        return LEVEL_WARNING
    else:
        return LEVEL_OK
```

Les seuils sont testés du plus grave au moins grave : l'ordre des conditions compte.

2.3 Les boucles (while, for)

Définition simple

Une boucle répète des instructions : **for** parcourt une collection, **while** recommence tant qu'une condition est vraie.

Analogie de la vie courante

Un distributeur de billets qui redemande le code PIN tant qu'il est faux.

Extrait 2 – cli.py — redemander le montant tant qu'il est invalide

```
def prompt_amount(self, prompt: str) -> float:
    # Concept: loop + decision -- re-prompt until the user enters a valid amount
    while True:
        raw = self.ask(prompt)
        try:
            return parse_amount(raw)
        except ValueError as exc:
            self.out(f" × {exc}. Please try again.")
```

Le menu principal est lui aussi une boucle **while self.running**; l'option 0 met **running** à **False**.

2.4 Les fonctions

Définition simple

Une fonction est un bloc de code nommé, qui reçoit des paramètres et renvoie un résultat. On l'écrit une fois, on l'utilise partout.

Analogie de la vie courante

Une recette de cuisine : écrite une fois, réutilisée à chaque repas, avec d'autres ingrédients.

`parse_amount` transforme "\$1,234.50" en 1234.5 et refuse ce qui n'est pas un montant valable. Elle sert pour les transactions, les budgets *et* les objectifs. Toutes les fonctions ont des annotations de type et une docstring au format Google (**Args**, **Returns**, **Raises**), vérifiées par un test.

2.5 Les listes

Définition simple

Une liste est une collection *ordonnée* d'éléments, qui peut contenir des doublons.

Analogie de la vie courante

Un relevé bancaire : les opérations dans l'ordre, et deux cafés identiques peuvent y figurer.

`FinanceTracker.transactions` est une liste ; les filtres construisent de nouvelles listes avec des compréhensions de liste.

2.6 Les dictionnaires

Définition simple

Un dictionnaire associe une *clé* à une *valeur* et retrouve la valeur à partir de la clé immédiatement.

Analogie de la vie courante

Un répertoire téléphonique : on cherche un nom, on obtient le numéro sans lire tout le carnet.

Les budgets sont indexés par catégorie, les objectifs par nom, et le menu de la console est un dictionnaire « touche → (libellé, méthode) » : ajouter une option tient en une ligne.

2.7 Les ensembles (`set`)

Définition simple

Un ensemble est une collection *sans doublons* et sans ordre, avec des tests d'appartenance très rapides.

Analogie de la vie courante

Une liste d'invités : ajouter deux fois « Alice » ne change rien.

Extrait 3 – `tracker.py` — la liste, les dictionnaires et l'ensemble dans le constructeur

```
def __init__(self, transactions: Iterable[Transaction] | None = None) -> None:
    # Concept: list -- transactions are ordered and may contain look-alike entries
    self.transactions: list[Transaction] = []
    # Concept: dictionary -- O(1) lookup of a budget/goal by its name
    self.budgets: dict[str, Budget] = {}
    self.goals: dict[str, SavingsGoal] = {}
    # Concept: set -- guarantees unique category names
    self.categories: set[str] = set(DEFAULT_EXPENSE_CATEGORIES | DEFAULT_INCOME_CATEGORIES)
    for transaction in transactions or []:
        self._store(transaction)
```

Autre usage : la différence d'ensembles trouve en une expression les colonnes manquantes d'un CSV (`required - set(reader.fieldnames)`).

2.8 Les classes et la programmation orientée objet

Définition simple

Une classe est un modèle qui regroupe des données (*attributs*) et les actions sur ces données (*méthodes*). Un objet est un exemplaire construit à partir de ce modèle.

Analogie de la vie courante

Le plan d'une maison (la classe) et les maisons construites (les objets), chacune avec ses pièces et ses usages.

Les modèles sont des *dataclasses* : Python écrit le constructeur, et la méthode `__post_init__` vérifie chaque champ juste après la création. Un objet invalide ne peut donc jamais exister.

Extrait 4 – models.py — validation d'une transaction à sa création

```
def __post_init__(self) -> None:
    self.date = parse_date(self.date)
    self.amount = parse_amount(self.amount)
    self.kind = str(self.kind).strip().lower()
    # Concept: decision structure -- reject anything outside the allowed kinds
    if self.kind not in VALID_KINDS:
        raise InvalidTransactionError(
            f"Invalid kind '{self.kind}': expected one of {sorted(VALID_KINDS)}"
        )
    self.category = normalize_category(self.category)
    if not self.category:
        raise InvalidTransactionError("Category must not be empty")
    self.description = str(self.description or "").strip()
    if len(self.description) > MAX_DESCRIPTION_LENGTH:
        raise InvalidTransactionError(
            f"Description must be at most {MAX_DESCRIPTION_LENGTH} characters"
        )
    if not self.id:
        self.id = _new_id()
```

2.9 La gestion des fichiers

Définition simple

Lire et écrire des fichiers pour que les données survivent à la fermeture du programme.

Analogie de la vie courante

Un classeur de bureau : on range les feuilles le soir, on les retrouve le lendemain.

`storage.py` utilise `pathlib` pour les chemins, `with open(...)` pour être sûr que le fichier se referme, et les modules `json` et `csv`. L'écriture est *atomique* : on écrit d'abord dans un fichier temporaire, puis on le met à la place du vrai d'un seul coup.

Extrait 5 – storage.py — écriture atomique avec try/except/finally

```
def _atomic_write(path: Path, write_func: Any, newline: str | None = None) -> None:
    path = Path(path)
    tmp_name: str | None = None
    # Concept: exception handling -- try/except/finally guarantees cleanup of the temp file
    try:
        path.parent.mkdir(parents=True, exist_ok=True)
        fd, tmp_name = tempfile.mkstemp(dir=path.parent, prefix=f"{path.name}.", suffix=".tmp")
        with os.fdopen(fd, "w", encoding=ENCODING, newline=newline) as handle:
            write_func(handle)
        # replace() is atomic on the same filesystem: readers see the old or new file,
        # never a partial one.
        Path(tmp_name).replace(path)
        tmp_name = None
    except OSError as exc:
        raise StorageError(f"Could not write '{path}': {exc.strerror or exc}") from exc
    finally:
        if tmp_name is not None:
            Path(tmp_name).unlink(missing_ok=True)
```

2.10 Les exceptions

Définition simple

Une exception signale une erreur pendant l'exécution. On peut l'*attraper* (`try/except`) pour réagir proprement au lieu de planter.

Analogie de la vie courante

L'airbag d'une voiture : quand un problème survient, il amortit le choc au lieu de laisser tout s'écraser.

Le projet définit sa propre hiérarchie d'erreurs : l'interface attrape `FinanceTrackerError` une seule fois et affiche le message, quelle que soit l'erreur précise.

Extrait 6 – `exceptions.py` — la hiérarchie complète des erreurs

```
class FinanceTrackerError(Exception):
    """Base class for every error raised by the finance tracker."""

class ValidationError(FinanceTrackerError, ValueError):
    """Raised when user-supplied data is invalid.

    Inherits from ``ValueError`` as well, so generic code that expects a
    ``ValueError`` for bad input keeps working.
    """

class InvalidTransactionError(ValidationError):
    """Raised when a transaction has an invalid amount, date, kind or category."""

class InvalidBudgetError(ValidationError):
    """Raised when a budget limit or category is invalid."""

class InvalidGoalError(ValidationError):
    """Raised when a savings goal has an invalid name, target or deadline."""

class TransactionNotFoundError(FinanceTrackerError, KeyError):
    """Raised when no transaction matches the requested id."""

    def __str__(self) -> str:
        """Return the plain message.

        KeyError wraps its message in quotes; a plain message is friendlier in the UI.

        Returns:
            The error message without surrounding quotes.
        """
        return str(self.args[0]) if self.args else "Transaction not found"

class StorageError(FinanceTrackerError, OSError):
    """Raised when data cannot be saved to or loaded from disk."""
```

2.11 pandas

Définition simple

pandas manipule des *tableaux de données* (DataFrame) : filtrer, regrouper, agréger, calculer des moyennes glissantes, en une ligne au lieu d'une boucle.

Analogie de la vie courante

Un tableur Excel automatisé : un seul `groupby` fait ce qu'on ferait à la main avec des filtres et des sous-totaux.

Extrait 7 – analytics.py — résumé mensuel par tableau croisé

```
def monthly_summary(df: pd.DataFrame) -> pd.DataFrame:
    columns = ["income", "expense", "net", "savings_rate"]
    if df.empty:
        return pd.DataFrame(columns=columns, index=pd.Index([], name="month"), dtype=float)
    # Concept: pandas -- group expenses by month and kind, then pivot kinds into columns
    summary = df.pivot_table(
        index="month", columns="kind", values="amount", aggfunc="sum", fill_value=0.0
    )
    summary = summary.reindex(columns=[INCOME, EXPENSE], fill_value=0.0)
    summary.columns = ["income", "expense"]
    summary["net"] = summary["income"] - summary["expense"]
    income = summary["income"].where(summary["income"] > 0)
    summary["savings_rate"] = (summary["net"] / income * 100).fillna(0.0)
    summary.columns.name = None
    return summary.round(2)
```

`pivot_table` met une ligne par mois et une colonne par type (revenu, dépense) ; le taux d'épargne est protégé contre la division par zéro (`where(income > 0)`).

3 Les bibliothèques

Pour chacune : à quoi elle sert, une analogie, où et comment elle est utilisée, et pourquoi ce choix plutôt qu'une alternative. Les versions sont celles réellement installées.

3.1 pandas (3.0.6)

Rôle. Toutes les analyses : résumé mensuel, catégories, tendance, plus grosses dépenses, indicateurs.

Analogie. Un tableur qui obéit à des formules écrites une fois. **Où.** `analytics.py`, puis le tableau de bord et la démo.

Extrait 8 – analytics.py — groupby avec agrégation nommée

```
def category_breakdown(df: pd.DataFrame, kind: str = EXPENSE) -> pd.DataFrame:
    subset = df[df["kind"] == kind]
    if subset.empty:
        return pd.DataFrame(columns=["category", "total", "share", "count"])
    # Concept: pandas -- groupby + named aggregation computes several stats at once
    grouped = (
        subset.groupby("category")
        .agg(total=("amount", "sum"), count=("amount", "size"))
        .sort_values("total", ascending=False)
        .reset_index()
    )
    grouped["share"] = grouped["total"] / grouped["total"].sum() * 100
    return grouped[["category", "total", "share", "count"]].round(2)
```

Pourquoi pandas. Le cours l'exige, et des boucles Python feraient le même travail en trois fois plus de lignes, avec plus de risques d'erreur. NumPy seul n'a pas d'étiquettes (mois, catégories).

3.2 matplotlib (3.11.2)

Rôle. Les graphiques PNG (figure 6) et ceux affichés dans la fenêtre Tkinter. **Analogie.** Une table à dessin : on place chaque élément soi-même. **Où.** `visualize.py`. Les figures sont créées avec `Figure()` et non `pyplot` : pas besoin d'écran (mode démo, tests), et la même figure s'intègre à Tkinter.

Extrait 9 – `visualize.py` — un dictionnaire « nom de fichier → figure »

```
def save_all_charts(
    df: pd.DataFrame, goals: Iterable[SavingsGoal], out_dir: Path = CHARTS_DIR
) -> list[Path]:
    # Concept: dictionary -- map each output file name to the function that draws it
    charts = {
        "category_pie.png": category_pie(df),
        "monthly_income_expenses.png": monthly_bars(df),
        "spending_trend.png": trend_line(df),
        "goals_progress.png": goals_progress(goals),
    }
    return [save_figure(fig, Path(out_dir) / name) for name, fig in charts.items()]
```

Pourquoi. C'est la référence Python, stable et hors ligne ; seaborn ajoute surtout du style, inutile ici.

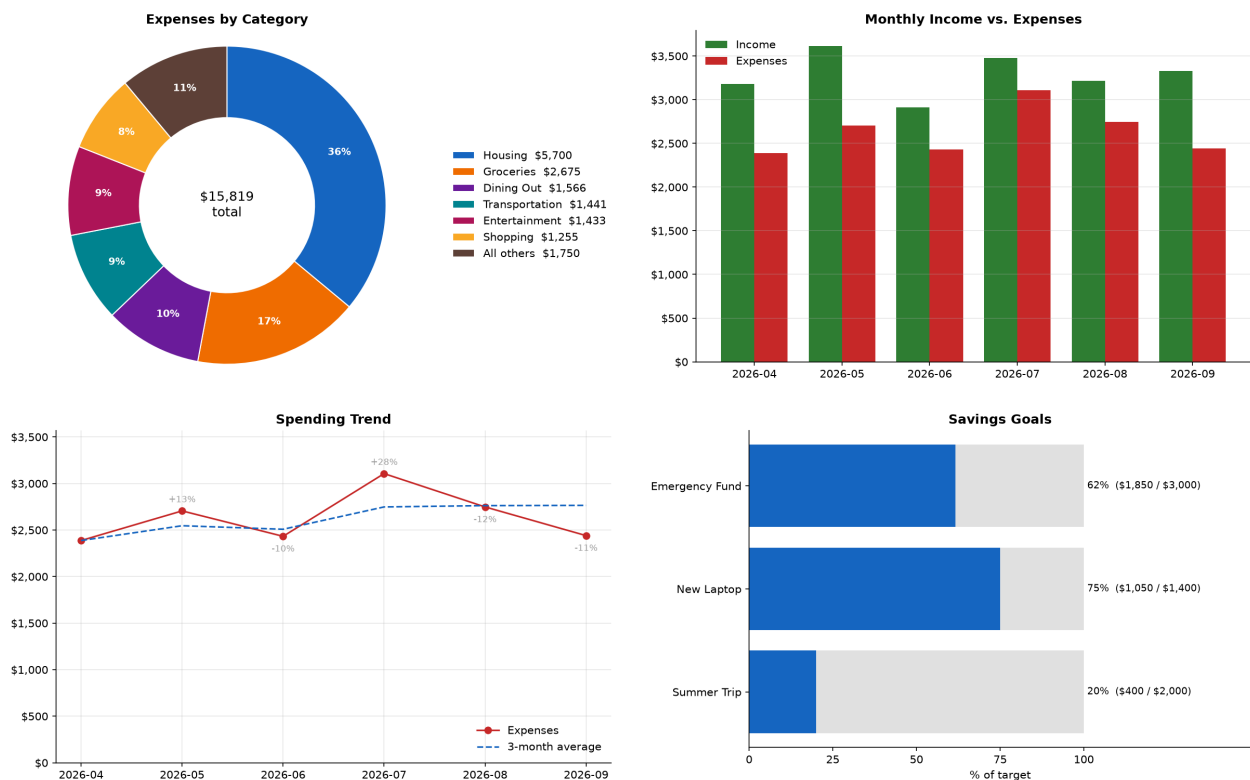


Figure 6 – Les quatre graphiques produits par python `main.py` demo dans `output/charts/` (copiés tels quels).

3.3 Plotly (7.1.0) et Streamlit (1.64.0)

Rôle. Streamlit transforme un script Python en page web ; Plotly fournit les graphiques interactifs (survol, zoom). **Analogie.** Streamlit est une vitrine qu'on réorganise en parlant ; Plotly, une carte

qu'on peut toucher. **Où.** `dashboard.py`. Le script est réexécuté de haut en bas à chaque clic ; `@st.cache_data` évite de relire le fichier si rien n'a changé.

Extrait 10 – `dashboard.py` — la fonction principale de la page

```
def main() -> None:
    st.set_page_config(page_title=PAGE_TITLE, page_icon="$", layout="wide")
    st.title("$ Personal Finance Dashboard")
    loaded = choose_data()
    if loaded is None:
        return
    tracker, label = loaded
    df_all = analytics.to_dataframe(tracker.transactions)
    if df_all.empty:
        st.warning("This file contains no transactions.")
        return
    start, end, categories, kind = sidebar_filters(df_all)
    # Concept: pandas -- the sidebar filters become boolean masks on the DataFrame
    df = analytics.filter_dataframe(df_all, start, end, categories)
    if kind == "Expenses":
        df = df[df["kind"] == EXPENSE]
    elif kind == "Income":
        df = df[df["kind"] == INCOME]
    st.caption(f"{label} · {start} → {end} · {len(df)} of {len(df_all)} transactions")
    if df.empty:
        st.warning("No transactions match the filters.")
        return
    kpi_row(df)
    tabs = st.tabs(["Overview", "Trends", "Budgets & Alerts", "Savings Goals", "Transactions"])
    with tabs[0]:
        overview_tab(df)
    with tabs[1]:
        trends_tab(df)
    with tabs[2]:
        budgets_tab(tracker)
    with tabs[3]:
        goals_tab(tracker)
    with tabs[4]:
        transactions_tab(df)
```

Pourquoi. Streamlit demande zéro HTML/JavaScript, contrairement à Flask ou Dash ; Plotly est intégré nativement à Streamlit.

3.4 Tkinter et ttk (Tk 8.6)

Rôle. L'application de bureau. **Analogie.** Un jeu de construction : fenêtres, onglets, champs, boutons qu'on assemble. **Où.** `gui_tkinter.py`, classe `FinanceApp` (elle hérite de `ttk.Frame`).

Extrait 11 – `gui_tkinter.py` — le bouton « Add transaction »

```
def add_transaction(self) -> None:
    # Concept: exception handling -- show validation errors in a dialog
    try:
        t = self.tracker.add_transaction(
            self.date_var.get(),
            self.amount_var.get(),
            self.kind_var.get(),
            self.category_var.get(),
            self.description_var.get(),
        )
    except FinanceTrackerError as exc:
        self._error(exc)
    return
    self.amount_var.set("")
```

```

self.description_var.set("")
self._update_category_choices()
self._changed(f"Added {t.kind} {money(t.amount)} ({t.category})")
if t.is_expense:
    for alert in alerts.check_budgets(self.tracker, t.month):
        if alert.subject == t.category:
            messagebox.showwarning("Budget alert", alert.message, parent=self.master)

```

Pourquoi. Tkinter est livré avec Python (aucune installation) et demandé par le cours ; PyQt est plus riche mais lourd et sous licence GPL/commerciale.

3.5 json, csv, pathlib (bibliothèque standard)

Rôle. json sauvegarde tout l'état (transactions, budgets, objectifs) ; csv échange les transactions avec Excel ; pathlib construit les chemins. **Analogie.** JSON est un classeur avec des intercalaires ; CSV, une simple feuille de calcul. **Où.** storage.py, config.py. **Pourquoi.** Formats lisibles par un humain et par d'autres programmes, sans dépendance. pickle serait illisible et dangereux à ouvrir ; SQLite serait excessif pour quelques centaines de lignes.

3.6 dataclasses, uuid, tempfile, argparse (bibliothèque standard)

dataclasses génère le code répétitif des classes de données ; uuid donne à chaque transaction un identifiant unique de 8 caractères ; tempfile et os servent à l'écriture atomique ; argparse lit la ligne de commande (cli | gui | demo, --data, --output).

3.7 pytest (9.1.1), pytest-cov (7.1.0) et ruff (0.16.9)

Rôle. pytest lance les tests, pytest-cov mesure quelles lignes ils exécutent, ruff vérifie et formate le style. **Analogie.** Un contrôle technique automatique avant chaque mise en circulation. **Pourquoi.** pytest est plus concis qu'unittest (simples assert, fixtures) ; ruff remplace flake8, isort et black en un seul outil très rapide.

3.8 Outils des livrables : rich (15.0.0), Playwright (1.63.0), python-docx, python-pptx

Ils ne servent pas à l'application mais à produire les documents de façon reproductible : rich enregistre la vraie sortie console, Playwright pilote un vrai navigateur pour capturer le tableau de bord, python-pptx génère les slides et python-docx le document CodeStepByStep.

4 Le travail en détail, module par module

4.1 Vue d'ensemble et flux des données

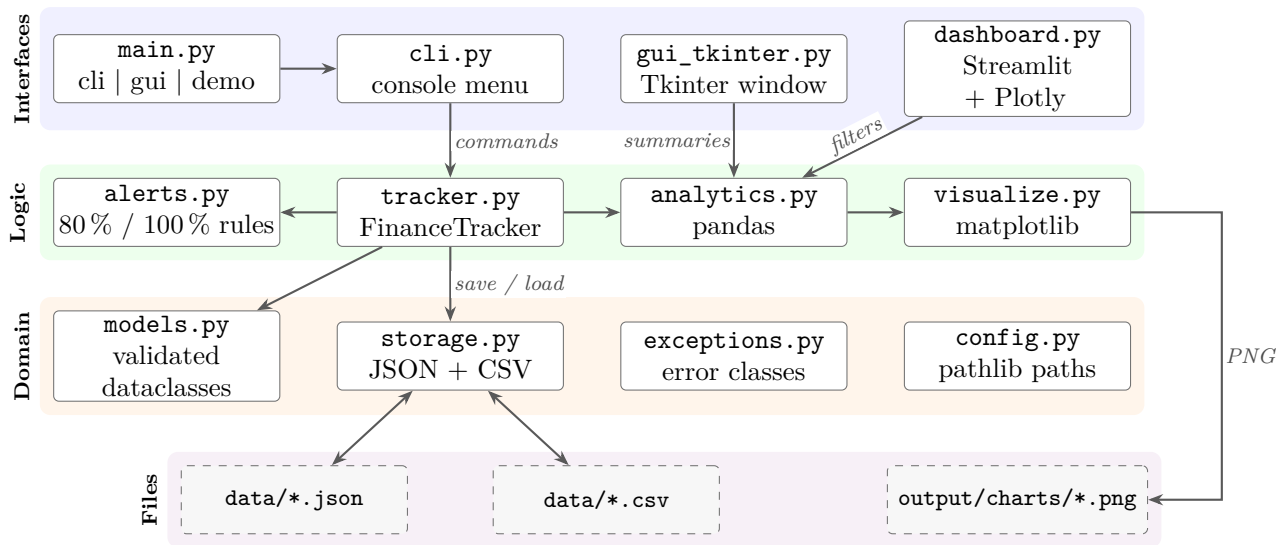


Figure 7 – Les modules et les flux de données. Les interfaces ne contiennent aucune règle métier : elles envoient des commandes au `FinanceTracker`, qui délègue les fichiers à `storage.py` ; `analytics` produit les DataFrames qui alimentent tableaux, indicateurs et graphiques.

Figure 8 suit le trajet d'une dépense, de la saisie à la sauvegarde.

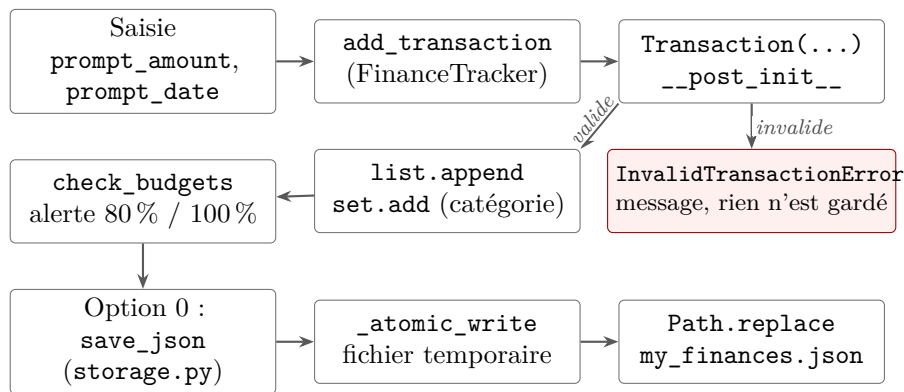


Figure 8 – La vie d'une dépense dans le mode console : saisie validée, création de l'objet (validation), stockage en mémoire, alerte immédiate, puis sauvegarde atomique à la sortie.

4.2 models.py — les fiches de base

Rôle. Décrire une transaction, un objectif d'épargne et un budget, et refuser toute valeur invalide. **Éléments clés.** `Transaction`, `SavingsGoal`, `Budget`, `parse_amount`, `parse_date`, `normalize_category`. **Erreurs.** Montant non numérique, nul, négatif, infini ou trop grand ; date au mauvais format ; type autre que `income/expense` ; catégorie vide : chaque cas lève une erreur précise.

Extrait 12 — models.py — arrondir avant de valider

```
def parse_amount(value: float | int | str) -> float:
    if isinstance(value, bool): # bool is a subclass of int -- reject it explicitly
        raise InvalidTransactionError("Amount must be a number")
```

```

try:
    # Users often type currency symbols and thousands separators; accept them.
    cleaned = str(value).replace("$", "").replace(",", "").strip()
    amount = float(cleaned)
except ValueError as exc:
    raise InvalidTransactionError(f"Invalid amount '{value}': not a number") from exc
if not math.isfinite(amount): # rejects NaN and infinity
    raise InvalidTransactionError("Amount must be a finite number")
# Round first: "0.004" would otherwise pass the check and be stored as 0.00.
amount = round(amount, 2)
if amount <= 0:
    raise InvalidTransactionError("Amount must be at least 0.01")
if amount > MAX_AMOUNT:
    raise InvalidTransactionError(f"Amount must not exceed {MAX_AMOUNT:,.0f}")
return amount

```

Point fin à connaître : l'arrondi se fait *avant* le test « supérieur à zéro ». Sinon "0.004" passait le test puis devenait 0,00 \$, ce qui provoquait plus tard une division par zéro (bogue trouvé par une revue indépendante, puis corrigé et testé).

4.3 exceptions.py — les erreurs du projet

Rôle. Donner un nom à chaque problème. **Idée clé.** L'héritage multiple : `InvalidTransactionError` est à la fois une `FinanceTrackerError` (attrapée par l'interface) et une `ValueError` (comprise par du code Python générique).

4.4 storage.py — lire et écrire les fichiers

Rôle. Sauvegarder et charger le JSON, exporter et importer le CSV. **Éléments clés.** `save_json`, `load_json`, `save_csv`, `load_csv`, `AppState`. **Gestion des erreurs.**

- fichier absent : ce n'est pas une erreur (premier lancement), on part de zéro ;
- JSON illisible *ou* mal structuré : le fichier est renommé `*.corrupted-<date>` (rien n'est perdu) et l'application démarre à vide ;
- enregistrement invalide ou identifiant en double : ignoré et signalé, les autres sont chargés ;
- CSV sans les colonnes obligatoires : erreur claire, avec la liste des colonnes manquantes.

Extrait 13 – `storage.py` — `try / except / else` pour récupérer un fichier corrompu

```

def load_json(path: Path | str, recover: bool = True) -> AppState:
    path = Path(path)
    state = AppState()
    if not path.exists():
        state.warnings.append(f"No data file at '{path}' -- starting with empty data.")
        return state

    # Concept: exception handling -- recover from a corrupted JSON file
    try:
        # Concept: file handling -- `with` closes the file even if json.load fails
        with path.open("r", encoding=ENCODING) as handle:
            payload = json.load(handle)
            _check_structure(payload)
    except (json.JSONDecodeError, UnicodeDecodeError, ValueError) as exc:
        if not recover:
            raise StorageError(f"Corrupted JSON file '{path}': {exc}") from exc
        backup = _backup_corrupted(path)
        state.warnings.append(
            f"'{path.name}' was corrupted ({exc}); a backup was saved as '{backup.name}'."
        )

```

```

except OSError as exc:
    raise StorageError(f"Could not read '{path}': {exc.strerror or exc}") from exc
else:
    # `else` runs only when the file was read and parsed without error.
    _fill_state(state, payload)
return state

```

Extrait 14 — storage.py — un JSON valide mais de mauvaise forme est traité comme corrompu

```

def _check_structure(payload: Any) -> None:
    if not isinstance(payload, dict):
        raise ValueError("top-level JSON value must be an object")
    for section in SECTIONS:
        if not isinstance(payload.get(section, []), list):
            raise ValueError(f"'{section}' must be a list")

```

4.5 tracker.py — le classeur central

Rôle. Contenir toutes les données en mémoire et offrir les opérations : ajouter, modifier, supprimer, rechercher, filtrer, totaux, budgets, objectifs, import/export.

Extrait 15 — tracker.py — ajout d'une transaction

```

def add_transaction(
    self,
    date_value: date | str,
    amount: float | str,
    kind: str,
    category: str,
    description: str = "",
) -> Transaction:
    transaction = Transaction(
        date=date_value,
        amount=amount,
        kind=kind,
        category=category,
        description=description,
    )
    return self._store(transaction)

```

Extrait 16 — tracker.py — filtre multi-critères avec continue

```

def filter(
    self,
    kind: str | None = None,
    categories: Iterable[str] | None = None,
    start: date | str | None = None,
    end: date | str | None = None,
    min_amount: float | None = None,
    max_amount: float | None = None,
) -> list[Transaction]:
    wanted = {normalize_category(c) for c in categories} if categories else None
    kind = kind.strip().lower() if kind else None
    start_date = parse_date(start) if start else None
    end_date = parse_date(end) if end else None
    results: list[Transaction] = []
    # Concept: loop + decision -- each `continue` skips a transaction failing one test
    for t in self.transactions:
        if kind and t.kind != kind:
            continue
        if wanted is not None and t.category not in wanted:
            continue
        if start_date and t.date < start_date:
            continue

```

```

    if end_date and t.date > end_date:
        continue
    if min_amount is not None and t.amount < min_amount:
        continue
    if max_amount is not None and t.amount > max_amount:
        continue
    results.append(t)
return sorted(results, key=lambda t: t.date)

```

La modification (`update_transaction`) construit un *nouvel* objet : si une valeur est invalide, l'ancienne transaction reste intacte.

4.6 analytics.py — les calculs pandas

Rôle. Transformer la liste de transactions en DataFrame, puis calculer résumés, répartitions, tendances et indicateurs. Chaque fonction accepte des données vides et renvoie alors un tableau vide mais bien formé : les interfaces n'ont jamais de cas particulier.

Extrait 17 – analytics.py — de la liste d'objets au DataFrame

```

def to_dataframe(transactions: Iterable[Transaction]) -> pd.DataFrame:
    # Concept: pandas -- create a DataFrame from a list of dictionaries
    df = pd.DataFrame([t.to_dict() for t in transactions], columns=COLUMNS)
    df["date"] = pd.to_datetime(df["date"])
    df["amount"] = df["amount"].astype(float)
    # Vectorised: one expression for the whole column instead of a Python loop.
    df["signed_amount"] = df["amount"].where(df["kind"] == INCOME, -df["amount"])
    df["month"] = df["date"].dt.strftime("%Y-%m")
    return df.sort_values("date", ignore_index=True)

```

Extrait 18 – analytics.py — tendance : mois continus, moyenne glissante, variation

```

def spending_trend(df: pd.DataFrame, window: int = DEFAULT_TREND_WINDOW) -> pd.DataFrame:
    expenses = df[df["kind"] == EXPENSE]
    if expenses.empty:
        return pd.DataFrame(
            columns=["expense", "rolling_avg", "mom_change_pct"],
            index=pd.Index([], name="month"),
            dtype=float,
        )
    trend = expenses.groupby("month")["amount"].sum().to_frame("expense")
    # A month with no expenses must still count as a month (with 0 spent); otherwise
    # "month over month" could silently compare January with April.
    all_months = pd.period_range(df["month"].min(), df["month"].max(), freq="M")
    trend = trend.reindex(all_months.strftime("%Y-%m"), fill_value=0.0)
    trend.index.name = "month"
    # Concept: pandas -- rolling window smooths out one-off spikes in spending
    trend["rolling_avg"] = trend["expense"].rolling(window=window, min_periods=1).mean()
    change = trend["expense"].pct_change() * 100
    # Growth from a $0 month is infinite, which is meaningless to display.
    trend["mom_change_pct"] = change.replace([float("inf"), float("-inf")], float("nan"))
    return trend.round(2)

```

Deux détails à savoir expliquer : un mois sans dépense compte pour 0 (sinon on comparerait janvier à avril) ; une hausse depuis 0 \$ est infinie, donc affichée « - ».

4.7 alerts.py — budgets et jalons

Rôle. Comparer les dépenses d'un mois à chaque budget, et signaler les jalons et les échéances des objectifs. Les seuils sont des constantes (`WARNING_THRESHOLD = 0.80`, `EXCEEDED_THRESHOLD = 1.00`) ; les alertes sont triées de la plus grave à la moins grave.

Extrait 19 – `alerts.py` — une alerte par budget qui mérite l'attention

```
def check_budgets(
    tracker: FinanceTracker, month: str | None = None, include_ok: bool = False
) -> list[Alert]:
    month = month or tracker.latest_month()
    if month is None:
        return []
    spent_by_category = tracker.spending_by_category(month)
    alerts: list[Alert] = []
    # Concept: loop + decision -- build one alert per budget that needs attention
    for category, budget in sorted(tracker.budgets.items()):
        spent = spent_by_category.get(category, 0.0)
        ratio = budget.usage(spent)
        level = budget_level(ratio)
        if level == LEVEL_OK and not include_ok:
            continue
        if spent == budget.monthly_limit:
            detail = "limit reached"
        elif level == LEVEL_EXCEEDED:
            detail = f"over by ${spent - budget.monthly_limit:,.2f}"
        else:
            detail = f"${budget.monthly_limit - spent:,.2f} left"
        alerts.append(
            Alert(
                level=level,
                subject=category,
                message=(
                    f"{category} ({month}): ${spent:,.2f} of ${budget.monthly_limit:,.2f} "
                    f"({ratio:.0%}) -- {detail}"
                ),
                ratio=ratio,
            )
        )
    return sorted(alerts, key=lambda a: (SEVERITY_ORDER[a.level], -a.ratio))
```

4.8 `visualize.py` — les graphiques

Rôle. Quatre fonctions, une par graphique, qui renvoient une `Figure`. **Pièges réglés.** Deux signes \$ dans un même texte sont interprétés par matplotlib comme une formule mathématique : ils sont échappés (`\$`). Au-delà de 7 parts, le camembert regroupe les petites catégories dans « All others ».

Extrait 20 – `visualize.py` — le camembert (donut)

```
def category_pie(df: pd.DataFrame, title: str = "Expenses by Category", dpi: float = DPI) ->
    Figure:
    breakdown = analytics.category_breakdown(df)
    if breakdown.empty:
        return _empty_figure(title, dpi=dpi)
    totals = breakdown.set_index("category")["total"]
    if len(totals) > MAX_PIE_SLICES:
        head = totals.iloc[: MAX_PIE_SLICES - 1]
        rest = pd.Series({"All others": totals.iloc[MAX_PIE_SLICES - 1 :].sum()})
        totals = pd.concat([head, rest])

    fig = Figure(figsize=FIGSIZE, dpi=dpi)
    ax = fig.add_subplot()
    wedges, _texts, _autotexts = ax.pie(
        totals.values,
        autopct="%1.0f%%",
        startangle=90,
        counterclock=False,
        pctdistance=0.78,
        colors=PALETTE[: len(totals)],
```

```

        wedgeprops={"width": 0.45, "edgecolor": "white"}, # donut: easier to compare
        textprops={"color": "white", "fontsize": 9, "weight": "bold"},
    )
    ax.legend(
        wedges,
        [f"{escape_dollars(name)} ${value:,.0f}" for name, value in totals.items()],
        loc="center left",
        bbox_to_anchor=(1.0, 0.5),
        frameon=False,
    )
    ax.text(0, 0, f"${totals.sum():.0f}\ntotal", ha="center", va="center", fontsize=12)
    ax.set_title(title, weight="bold")
    ax.axis("equal")
    fig.tight_layout()
    return fig

```

4.9 config.py — les chemins

Tous les chemins sont calculés à partir de l'emplacement du fichier avec `pathlib` (`PROJECT_ROOT`, `DATA_DIR`, `CHARTS_DIR`...) : le projet fonctionne depuis n'importe quel dossier et sur n'importe quel système, sans chemin absolu codé en dur.

4.10 cli.py — le menu console

Rôle. La boucle de menu et une méthode par action. **Idée clé.** Les fonctions d'entrée et de sortie sont des paramètres (`input_func`, `output_func`) : les tests pilotent des sessions entières. Ctrl+D ou Ctrl+C déclenchent une sortie propre (avec sauvegarde), et si la sauvegarde échoue, on demande avant de quitter.

Extrait 21 – cli.py — la boucle de menu et la dernière ligne de défense

```

def run(self) -> None:
    self.running = True
    # Concept: loop -- the menu repeats until "quit" sets running to False
    while self.running:
        self.show_menu()
        try:
            choice = self.ask("Choose an option: ")
            entry = self.menu.get(choice)
            # Concept: decision structure -- unknown keys get a hint, not a crash
            if entry is None:
                self.out("    × Unknown option, please pick one from the menu.")
                continue
            entry[1]()
        except KeyboardInterrupt:
            self.out("")
            self.quit()
        except FinanceTrackerError as exc:
            # Last line of defence: any app error is reported and the menu continues.
            self.out(f"    × {exc}")

```

4.11 gui_tkinter.py — l'application de bureau

Rôle. La classe `FinanceApp` construit quatre onglets, remplit les tableaux et réagit aux clics. **Erreurs.** Toute erreur de validation s'affiche dans une boîte de dialogue; rien n'est enregistré. **Écrans haute résolution.** La hauteur des lignes, la largeur des colonnes et la résolution des graphiques sont calculées à partir de la taille réelle de la police, sinon les lignes se chevauchent.

4.12 dashboard.py — le tableau de bord

Rôle. Barre latérale (source, période, catégories, type), indicateurs avec variation sur le mois précédent, onglets Overview, Trends, Budgets & Alerts, Savings Goals, Transactions. **Erreurs.** Un fichier importé illisible affiche une erreur ; un fichier utilisateur corrompu n'est *jamais* renommé par le tableau de bord, qui ne fait que lire.

Extrait 22 – dashboard.py — échapper \$ pour le Markdown

```
def md_escape(text: str) -> str:
    return text.replace("$", "\\$")
```

4.13 main.py — le point d'entrée

Extrait 23 – main.py — choisir le mode selon la ligne de commande

```
def main(argv: list[str] | None = None) -> int:
    args = build_parser().parse_args(argv)
    # Symbols such as "✓" or "■" cannot be encoded by some Windows consoles; replace
    # them with "?" instead of crashing with UnicodeEncodeError.
    if hasattr(sys.stdout, "reconfigure"):
        sys.stdout.reconfigure(errors="replace")
    # Concept: decision structure -- one branch per execution mode
    if args.mode == "cli":
        from finance_tracker import cli

        cli.run(args.data or USER_DATA_FILE)
    elif args.mode == "gui":
        # Imported lazily so that cli/demo work even where Tk is not installed.
        from finance_tracker import gui_tkinter

        gui_tkinter.run(args.data or USER_DATA_FILE)
    else:
        data_file = args.data or SAMPLE_JSON
        if not data_file.exists():
            print(f"Error: data file not found: {data_file}", file=sys.stderr)
            return 1
        run_demo(data_file, args.output)
    return 0
```

Les imports de `cli` et `gui_tkinter` sont faits à l'intérieur des branches : le mode démo fonctionne même sur une machine sans Tk.

5 Les données

5.1 Le format CSV

Une ligne par transaction, six colonnes ; les colonnes obligatoires sont `date`, `amount`, `kind` et `category`. Début réel de `data/sample_transactions.csv` :

Extrait 24 – data/sample_transactions.csv (5 premières lignes)

```
id,date,amount,kind,category,description
c9af18f8,2026-04-01,1513.94,income,Salary,Paycheck
96fc734d,2026-04-01,950.0,expense,Housing,Rent (shared apartment)
1d61fac3,2026-04-03,25.77,expense,Groceries,Farmers market
d3016989,2026-04-03,26.24,expense,Dining Out,Coffee
```

5.2 Le format JSON

Le JSON contient tout l'état : transactions, budgets, objectifs, plus un numéro de version et la date de sauvegarde. Extrait réel de `data/sample_data.json` (2 transactions, 1 budget, 1 objectif) :

Extrait 25 – Extrait de `data/sample_data.json`

```
{
  "version": 1,
  "saved_at": "2026-09-30T15:06:44",
  "transactions": [
    {
      "id": "c9af18f8",
      "date": "2026-04-01",
      "amount": 1513.94,
      "kind": "income",
      "category": "Salary",
      "description": "Paycheck"
    },
    {
      "id": "ca8f3653",
      "date": "2026-04-15",
      "amount": 1452.5,
      "kind": "income",
      "category": "Salary",
      "description": "Paycheck"
    }
  ],
  "budgets": [
    {
      "category": "Groceries",
      "monthly_limit": 500.0
    }
  ],
  "goals": [
    {
      "name": "Emergency Fund",
      "target": 3000.0,
      "saved": 1850.0,
      "deadline": "2027-06-30"
    }
  ]
}
```

5.3 Pourquoi ces choix

Les montants sont toujours positifs, et c'est le champ `kind` qui dit si c'est un revenu ou une dépense : la validation est plus simple. Les dates sont au format ISO (AAAA-MM-JJ), qui se trie correctement et que pandas lit directement.

5.4 La génération des données d'exemple

`tools/generate_sample_data.py` crée 311 transactions *fictives* sur 6 mois (2026-04 à 2026-09) pour un étudiant imaginaire : salaire toutes les deux semaines, missions freelance, loyer et abonnements fixes, dépenses variables (courses, restaurants, transports...), un pic de dépenses en juillet pour rendre la tendance intéressante, 7 budgets et 3 objectifs. Une graine aléatoire fixe (42) rend chaque génération identique, y compris les identifiants : les tests, les captures et la vidéo montrent toujours les mêmes chiffres.

6 Les tests

6.1 Ce qui est testé, et pourquoi

Le projet compte **175 tests automatiques**. La couverture est de **98 %** des lignes du paquet `finance_tracker`, hors `gui_tkinter.py` et `dashboard.py` : ces deux fichiers sont surtout du code d'événements graphiques, vérifié par des tests de fumée (*smoke tests*).

- **Modèles** : chaque règle de validation, y compris les cas limites ("0.004", NaN, infini, booléen, catégorie vide).
- **Stockage** : allers-retours JSON/CSV, fichier absent, corrompu, mal structuré, colonnes manquantes, lignes invalides, identifiants en double, BOM d'Excel.
- **Tracker, alertes, analyses** : chaque calcul pandas est comparé à un résultat calculé à la main sur un petit jeu de données de deux mois.
- **Console** : des sessions complètes pilotées par des réponses préparées, avec saisies invalides.
- **GUI et tableau de bord** : la fenêtre Tkinter est construite et cliquée sur un vrai affichage ; la page Streamlit tourne sans navigateur avec AppTest.
- **Standards** : un test vérifie les en-têtes de module, les docstrings Google, les annotations de type et la présence de chaque concept du cours.

Extrait 26 – `tests/test_storage.py` — un exemple de test réel

```
def test_corrupted_json_is_backed_up(tmp_path: Path) -> None:
    path = tmp_path / "data.json"
    path.write_text('{"transactions": [ {"date": ', encoding="utf-8")

    state = storage.load_json(path)

    assert state.transactions == []
    assert "corrupted" in state.warnings[0]
    assert not path.exists()
    backups = list(tmp_path.glob("data.json.corrupted-*"))
    assert len(backups) == 1
    assert backups[0].read_text(encoding="utf-8").startswith('{"transactions"')
```

6.2 Comment les lancer

Extrait 27 – Lancer les tests et les contrôles de style

```
pytest -q                # tous les tests
pytest --cov             # avec la couverture
ruff check . && ruff format --check .
```

7 Commandes utiles

Extrait 28 – Installation (Python 3.11+ requis)

```
git clone https://github.com/14juanito/Personal-finance-tracker.git
cd Personal-finance-tracker
python3 -m venv .venv
source .venv/bin/activate      # Windows : .venv\Scripts\activate
pip install -r requirements.txt
python -m playwright install chromium # seulement pour les captures
```

Extrait 29 – Lancer l'application

```
python main.py demo          # tout, sans interaction
python main.py cli           # menu console
python main.py gui           # fenêtre Tkinter
streamlit run finance_tracker/dashboard.py
```

Extrait 30 – Régénérer les données et les livrables

```
python tools/generate_sample_data.py    # données d'exemple (graine 42)
python tools/capture_screenshots.py     # vraies captures -> docs/figures/
python tools/build_latex.py             # rapport + ce mémo -> deliverables/*.pdf
python tools/build_slides.py            # Demo_Slides.pptx
python tools/build_screenshot_doc.py    # document CodeStepByStep (14 captures)
python tools/build_source_zip.py        # archive du code source
```

8 Les 15 questions probables de l'évaluateur

1. Pourquoi des dataclasses plutôt que des dictionnaires ?

Une dataclass garantit la présence de tous les champs et permet de valider les données dans `__post_init__`. Elle ajoute des méthodes (`signed_amount`, `to_dict`). Avec un dictionnaire, une faute de frappe dans une clé passerait inaperçue.

2. Pourquoi une liste pour les transactions, mais un dictionnaire pour les budgets ?

Les transactions ont un ordre et peuvent se ressembler : une liste convient. Un budget se cherche par catégorie : le dictionnaire donne un accès direct et empêche d'avoir deux budgets pour la même catégorie.

3. À quoi sert l'ensemble des catégories ?

À garantir l'unicité automatiquement. En plus, `normalize_category` transforme « food », « Food » et « FOOD » en « Food » : ces variantes ne créent pas trois catégories.

4. Que se passe-t-il si le fichier JSON est corrompu ?

`load_json` attrape l'erreur, renomme le fichier en `*.corrupted-<horodatage>` (rien n'est perdu), ajoute un avertissement et renvoie un état vide : l'application démarre normalement. Un JSON valide mais mal structuré (`{"transactions": null}`) est traité de la même façon. Un test le prouve : `test_corrupted_json_is_backed_up (tests/test_storage.py)`.

5. Et si un seul enregistrement est invalide ?

Le chargement a un `try/except` par enregistrement : la ligne fautive est ignorée et signalée, les autres sont chargées.

6. Pourquoi une écriture atomique ?

Si le programme plante au milieu d'une sauvegarde, un fichier écrit directement serait à moitié vide. On écrit dans un fichier temporaire, puis `Path.replace()` le substitue en une seule opération. Le bloc `finally` supprime le fichier temporaire en cas d'erreur.

7. Pourquoi JSON *et* CSV ?

Le JSON garde tout l'état, y compris les données imbriquées (budgets, objectifs). Le CSV est un tableau plat, idéal pour Excel ou pandas, mais il ne contient que les transactions.

8. Comment fonctionnent les alertes de budget ?

Pour le mois choisi, on calcule « dépensé / limite ». À partir de 0,80 c'est WARNING, à partir de 1,00 EXCEEDED. Les alertes sont triées de la plus grave à la moins grave, et la console comme la fenêtre en affichent une dès qu'une dépense franchit un seuil.

9. Explique le résumé mensuel avec pandas.

`pivot_table(index="month", columns="kind", values="amount", aggfunc="sum")` donne une ligne par mois et une colonne par type. On en déduit le solde et le taux d'épargne, avec une protection contre la division par zéro quand il n'y a pas de revenu.

10. Qu'est-ce que la moyenne glissante, et pourquoi l'utiliser ?

`rolling(window=3).mean()` fait la moyenne des 3 derniers mois. Elle lisse un pic ponctuel (juillet, +28%) pour montrer la vraie tendance. `pct_change()` donne la variation d'un mois sur l'autre.

11. Comment as-tu testé un programme interactif ?

`ConsoleApp` reçoit ses fonctions d'entrée et de sortie en paramètres ; les tests lui passent une liste de réponses et vérifient l'affichage. La fenêtre Tkinter a un test de fumée sur un vrai affichage, et le tableau de bord est testé avec `AppTest` de Streamlit.

12. Quel a été ton bogue le plus intéressant ?

Les montants du type « \$1,850 / \$3,000 » perdaient leurs signes dollar et passaient en italique : matplotlib, comme le Markdown de Streamlit, lit le texte entre deux \$ comme une formule mathématique. Les tests ne l'ont pas vu ; c'est la capture d'écran qui l'a révélé. Corrigé en échappant le signe, avec un test de non-régression.

13. Pourquoi l'ordre des `except` compte-t-il ?

`StorageError` hérite de `OSError`. Dans `load_csv`, un `except OSError` générique attrapait donc notre propre erreur et masquait le vrai message. Python teste les `except` dans l'ordre, et un parent attrape ses enfants : on a ajouté `except StorageError: raise` avant.

14. Pourquoi l'interface ne contient-elle pas de logique métier ?

Pour réutiliser et tester. Les trois interfaces appellent les mêmes fonctions : les chiffres sont identiques partout, et la logique se teste sans écran. Ajouter le tableau de bord n'a demandé aucune modification du cœur.

15. Que pourrais-tu améliorer ?

Des transactions récurrentes automatiques (loyer, abonnements), l'import de relevés bancaires avec suggestion de catégorie, une base SQLite pour de gros historiques, et des prévisions de dépenses dans le tableau de bord.

9 Glossaire

Terme	Définition simple
Agrégation	Résumer plusieurs lignes en une valeur (somme, moyenne, nombre).
Alerte de budget	Message affiché quand les dépenses d'une catégorie atteignent 80 % (WARNING) ou 100 % (EXCEEDED) de sa limite mensuelle.
Annotation de type	Indication du type attendu (<code>amount: float</code>), lue par les outils et les humains.
Atomique (écriture)	Écriture qui réussit entièrement ou pas du tout : jamais de fichier à moitié écrit.
Couverture de code	Pourcentage des lignes exécutées par les tests.
CSV	Fichier texte en tableau, une ligne par enregistrement, valeurs séparées par des virgules.
DataFrame	Tableau pandas : lignes, colonnes nommées, calculs vectorisés.
Dataclass	Classe de données dont Python écrit automatiquement le constructeur.
Docstring	Texte de documentation placé au début d'une fonction ou d'une classe.
Exception	Signal d'erreur pendant l'exécution, que l'on peut attraper avec <code>try/except</code> .
Fixture (pytest)	Donnée de test préparée une fois et fournie automatiquement aux tests.
groupby	Regrouper les lignes selon une colonne, puis calculer sur chaque groupe.
Héritage	Une classe enfant reprend tout ce que fait sa classe parente et peut l'étendre.
JSON	Format texte de données structurées (objets, listes), lisible par l'humain.
Moyenne glissante	Moyenne recalculée sur une fenêtre qui avance (ici 3 mois).
pathlib	Module standard qui manipule les chemins de fichiers comme des objets.
pivot_table	Tableau croisé : lignes, colonnes et valeur agrégée choisies.
Pseudo-terminal	Terminal simulé ; sert à enregistrer une vraie session console.
Smoke test	Test rapide qui vérifie qu'une interface démarre et réagit, sans tout détailler.
Taux d'épargne	$(\text{revenus} - \text{dépenses}) / \text{revenus}$, en pourcentage.
Tkinter / ttk	Bibliothèque graphique livrée avec Python ; ttk fournit des widgets modernes.
Treeview	Widget Tkinter qui affiche un tableau à colonnes.
Validation	Vérifier qu'une donnée est correcte avant de l'accepter.
Xvfb	Écran virtuel sous Linux, utilisé pour capturer la fenêtre sans afficher l'écran réel.